University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Data Structures Section D20
Session: Spring 2026

Assignment no Five:
😊 **DON'T PANIC** 😊
Due on Portal Only
Submission Before WED 11:59 PM June 10, 2026
The Use of ChatGPT is Allowed for Understanding
Copy (Even One line) from ChatGPT or Anyone or Anywhere Else is Zero Marks
100 Marks
Final Marks are based on Assignment Evaluation
You Should be Able to Write the Submitted Code in Evaluation

Topic	Recursion
Objective/ Outcome	• Understanding BinaryTree • Understanding BinarySearchTree • Applications of BinarySearchTree
5 Tasks	**BinarySerachTree**

CLO	Description	Taxonomy Level	PLO
2	Solve basic problems using fundamental concepts of programming language (C++).	C3 (Apply)	2

General Instructions

- 😊 **DON'T PANIC** 😊
- Make sure to read the **whole assignment document** carefully and take **notes in your DS Notebook**
- **DON'T DISREGARD INSTRUCTIONS**
- Recommended to submit work at least one hour before the deadline to avoid any internet or portal issues
- **CORRECTION** of any **FOUND ERRORS** or **MISTAKES** is **part of the assignment and Result in Bonus Marks**

DON'T ATTEMP THE SOLUTION BEFORE CAREFUL READING OF THE ASSIGNMENT DESCRIPTION (and taking notes in the DS notebook)

(Your DS Notebook may be checked for Assignment 4 Evaluation)

MARKS ARE BASED ON EFFORT AND NOT CORRECTION

- **Cheating or Copy is Cheating Yourself out of Learning**
- **Cheating or Copy may give you undeserved marks in the assignment but you lose in quizzes, exams and life**
- Use of ChatGPT is Encouraged for help and learning but **PLEASE DON'T COPY**
- Discussion with fellow Students is Encouraged for help and learning but **PLEASE DON'T COPY**
- **So please don't copy from anyone or anywhere**

SUBMISSION DEADLINE EXTENSION POLICY

- **Extension** in the **deadline** is **possible** with some **penalty (10 – 25 marks deduction) but must** submit at least **Half the Tasks** by **the deadline on the portal or NO** extension and no marks

Submission of Complete Assignment by **the deadline on the portal** will earn **25 % bonus Marks (Bonus Marks can be redeemed for future Assignment Marks Deficiencies)**

PORTAL SUBMISSION INSTRUCTIONS

- **For each task,** please Submit **one single code file** named "t#.cpp" (**must use the following provided template and the provided Classes**), **So for task 1 the file name will be t1.cpp and so on**, so for complete assignment you will need to submit 4 .cpp files and on set of all the needed .h files for each of the task to run
- **Please submit the header files used to compile these .cpp files, otherwise the code will not compile**
- **Write Solution in English in Comments in each .cpp File (See example template below). The C++ code has no marks without this English Solution Description.** The **English Solution** must have have formal itemized formal steps (See example in template below)
- **Zip** or **RAR** files have **ZERO** Marks
- **Submissions by email will not be considered and graded**

MARKING SCHEME

- **No English Description of solution No marks as it will be assumed that the code is not your own**
- **Code does not compile or run 50 % deducted (make sure to submit all needed header files)**

Example Template File for Task 1:

```
// SPRING 2026 DATA STRUCTURES SECTION D20 ASSIGNMENT FIVE
// Name of Student:
// Registration of Student:
// File t1.cpp contains complete code of task one, that will compile and run
// Must use the following includes only, any other include must be justified in comments or will
disqualify the solution
#include <iostream>
using namespace std;
#include "Stack.h"
#include "MyStack.h"
#include "Queue.h"
#include "MyQueue.h"
/***** TASK DESCRIPTION *****/
// Copy task # (e.g. 1) description here (see the example below)
/*
        Read 5 integers from the user, calculate and display average and maximum value. You are
        allowed to use Stack ADT only and Queue only and no other ADT
*/
/***** TASK SOLUTION DESCRIPTION *****/
// Write the Steps of Solution here (follow the same format and style of the example below)
/*
Stack Only Solution without functions
--------------------------------------------------
        1. Create a Stack of S Size 5     -----
                                          -----
        2. read values from the user and push on Stack until Stack is Full

                23
                -----
                29
                -----
                -23
                -----
                12
                -----
                -100
                -----
        Stack: currentIndex = 4 and count = 5

        3. Use Copy Constructer to make a Copy of Stack to preserve. Copy Stack CS = S
        4. create 2 variables sum and max and initialize with CS.pop()
        5. Until CS is empty pop from CS and save in a variable value
                6. Add value to sum
```

7. compare values with max and save the largest in max
8. increment a counter to get the number of values
8. display max and sum/counter;

Stack Only Solution functions

Function to Use:

1) **Read**
2) **Stats**

1. Create a Stack of S Size 5 -----
-----

2. Call function read

23

29

-23

12

-100

Stack: currentIndex = 4 and count = 5

3. Use Copy Constructer to make a Copy of Stack to preserve. Copy Stack CS = S
4. create 2 variables sum and max and initialize with CS.pop()
5. Call function Stats

Queue Only Solution without any functions

--

1. Create a Queue of Q Size
2. read values from the user and enqueue in Queue until Queue is Full
Q: 22 → 345 → -55 → 100 → 234
3. Use Copy Constructer to make a Copy of Queue to preserve. Copy Queue CQ = Q
CQ: 22 → 345 → -55 → 100 → 234
4. create 2 variables sum and max and initialize with CQ.dequeue()
5. Until CQ is empty dequque from CQ and save in a variable value
6. Add value to sum
7. compare values with max and save the largest in max
8. increment a counter to get the number of values
8. display max and sum/counter;

Queue Only Solution functions

```
---------------------------------------------------------
Function to Use:
    1)  Read
    2)  Stats

        1. Create a Queue of Q Size
        2. Call Read
                Q: 22 → 345 → -55 → 100 → 234
        3. Use Copy Constructer to make a Copy of Queue to preserve. Copy Queue CQ = Q
                CQ: 22 → 345 → -55 → 100 → 234
        4. create 2 variables sum and max and initialize with CQ.dequeue()
        5. Call Stats
*/


/***** GLOBAL FUNCTION PROTOTYPES *****/
// Write PROTOTYPES of All Global Functions Here

/***** Driver Function Main *****/
// CODE OF main here

/***** GLOBAL FUNCTION CODE *****/
// Write Codes of All Global Functions Here
```

BinaryTree.h and BinarySearchTree.h files:

(must not change these files unless you write a valid justification in comments and explain the change)

The zip file posted on portal have all the needed .h files. These files are tested to compile and run on Visual Studio 2022

Therefore, there should be only 4 .CCP files one for each task along with the above two .h files

Any other submitted file will not be graded

Task 1: BinaryTree and BinarySearchTree

Please write C+ code of all the public functions of **BinaryTree and BinarySearchTree,** and use a suitable main to test these functions:(**All functions must be Iterative even any Helper function that you may use**)

Task 2: Problems from Book

Please write the code of the following functions from the textbook and use a suitable main to test these functions:

1. Function, swapSubtrees, that swaps all of the left and right subtrees of a binary tree.
2. Function, singleParent, that returns the number of nodes in a binary tree that have only one child.
3. Function, convert2List that inserts the nodes of a binary tree into an ordered linked list.

4. Function, split that splits a Binary Tree in left subtree and right subtree from root
5. Function, subtree that returns a sub tree of a Binary Tree from a Matching Value

Task 3: BinarySerachTree Application

Main application of the BinarySearchTree is to store values for quick search. In this task, you will write a complete program that does the following:

1. Read text from a text file and stores the words in a BinarySearchTree. (when adding to the tree, lowercase and UPPERCASE character are treated as different
2. Write a BinarySearchTree of words to a text file
3. Able to search a word from the BinarySearchTree
4. Able to delete a word in the BinarySearchTree
5. Able to insert a new word in the BinarySearchTree
6. Able to process the words in the BinarySearchTree: Convert lowercase characters to UPPERCASE and vice-versa (after the conversion any duplicated words must be removed)

Task 4: BinarySerachTree Application

In a Blockchain e.g. Bitcoin, each node or coin in the chain has an alphanumeric key (mixture of digits and characters)that is stored in a public distributed register. When a new coin is mined then a new key is generated and added to the register, then the chain is verified by at least 5 other nodes to have the correct keys. Let us assume that a Blockchain is implemented as a BinarySearchTree and the corresponding distributed register also stores the keys in a BinarySearchTree. A Node of the Blockchain tree is have these attributes: public alphanumeric ID, private value, key (to match with the key in the key register) and password.
In this task you will implement the Blockchain tree and the tree of alphanumeric keys, by providing the following functionality

1. Add a new Node to the Blockchain tree and the corresponding key to the key register tree.
2. Search the Blockchain using the key, first key is verified in the key register and then searched in the Blockchain
3. Delete the Node by key, both key register and Blockchain Nodes are deleted
4. Verify a key, by checking all keys in the key register and then all the Nodes in the Blockchain
5. Display that displays in descending order the keys from key register and the Node data from Blockchain

Task 5: N-Order Tree

In this task you will use the following Node for the N-Order Tree.

```
template <class T>
class Node
{
        T data;
        Node * children;
        int noOfChildren;
};
```

Using the BinarySearchTree as an example, please do the following:

1. Declare and implement functions, corresponding to BinarySearchTree , of a NSearchTree. The children Nodes must have values in Ascending order. Half the children will have values less than the parent and half more than the parent
2. Write a main to test your code of the NSearchTree